

บทที่ 2

ทฤษฎีพื้นฐานเกี่ยวกับเว็บเซอร์วิส

2.1 เว็บเซอร์วิสคืออะไร

เว็บเซอร์วิส คือ แอปพลิเคชันหรือซอฟต์แวร์ที่ทำงานอย่างใดอย่างหนึ่งในลักษณะให้บริการ โดยจะถูกเรียกใช้งานจากแอปพลิเคชันอื่นๆ ผ่านทางหน้าเว็บ ในรูปแบบ RPC (Remote Procedure call) โดยที่ภาษาที่ใช้ในการติดต่อสื่อสารระหว่างคอมพิวเตอร์คือภาษาเอ็กซ์เอ็มแอล (XML)

การอธิบายวิธีการใช้งานของเว็บเซอร์วิส นั้น จะอธิบายโดยใช้ภาษาวิสเดิล WSDL (Web Service Description Language) ซึ่งเป็นภาษา XML ประเภทหนึ่ง

ระบบอื่นๆ จะสามารถติดต่อและทำงานกับเว็บเซอร์วิสได้ โดยใช้โปรโตคอลที่ชื่อว่า SOAP (Simple Object Protocol) ซึ่งใช้ภาษา XML เป็นมาตรฐานในการติดต่อระหว่างระบบผ่านทางโปรโตคอลอื่นที่ใช้ในการส่งข้อมูลบนเว็บ อาทิเช่น โปรโตคอล HTTP

2.2 เปรียบเทียบระหว่างเว็บแอปพลิเคชันกับเว็บเซอร์วิส

ทั้งเว็บเซอร์วิสและเว็บแอปพลิเคชันต่างใช้โปรโตคอล HTTP หรือ อินเทอร์เน็ต เป็นช่องทางในการสื่อสารเหมือนกัน แต่มีวัตถุประสงค์ต่างกัน โดย เว็บแอปพลิเคชันใช้เพื่อการแลกเปลี่ยนไฟล์ HTML ระหว่างเว็บเซิร์ฟเวอร์ แต่ เว็บเซอร์วิสเป็นการแลกเปลี่ยน “บริการ” (ก้อนโปรแกรม Software Components) ระหว่างระบบสารสนเทศ ผ่านเว็บเซิร์ฟเวอร์

ในเรื่องของความสามารถโดยส่วนใหญ่จะใช้เว็บแอปพลิเคชัน ในการติดต่อกับผู้ใช้ผ่านทาง Internet browser เพื่อนำเสนอข้อมูลและการทำงานต่างๆ ส่วนเว็บเซอร์วิส จะทำหน้าที่ในการติดต่อกับเว็บเซิร์ฟเวอร์ เพื่อแลกเปลี่ยนข้อมูลและการทำงานหรือให้บริการข้ามระบบกันโดยใช้เว็บแอปพลิเคชัน หรือ Application Interface ในการติดต่อกับผู้ใช้ นอกจากนี้ เว็บเซอร์วิสยังสามารถทำงานกับระบบต่างๆ ได้มากกว่า 1 ระบบ ในขณะที่เว็บแอปพลิเคชันไม่สามารถทำได้โดยตรง ซึ่งสามารถสรุปการเปรียบเทียบได้ดังตาราง

หัวข้อ	เว็บเซอร์วิส	เว็บแอปพลิเคชัน
การเชื่อมต่อ	program-program	human-program
ภาษาที่ใช้	XML	HTML, XML
รายชื่อการให้บริการ	ค้นหาผ่าน UDDI	ค้นหาผ่าน search engine
ขอบเขตการใช้งาน	Business-to-Business (B2B) Business-to-Customer (B2C)	Business-to-Customer (B2C)

หัวข้อ	เว็บเซอร์วิส	เว็บแอปพลิเคชัน
โปรโตคอล(Protocol)	SOAP+HTTP	HTTP

ตารางที่ 2.1 การเปรียบเทียบระหว่าง เว็บแอปพลิเคชันกับเว็บเซอร์วิส

หลังจากที่ได้ทำความเข้าใจความหมายและข้อเปรียบเทียบระหว่างเว็บแอปพลิเคชันและเว็บเซอร์วิสแล้ว ความแตกต่างระหว่างเว็บทั้งสองจะเห็นได้ชัดในเรื่องของความสามารถ แต่ในการนำไปใช้งานจำเป็นจะต้องประยุกต์ใช้เว็บทั้งสองชนิดร่วมกัน เพื่อให้เกิดประสิทธิภาพมากที่สุด

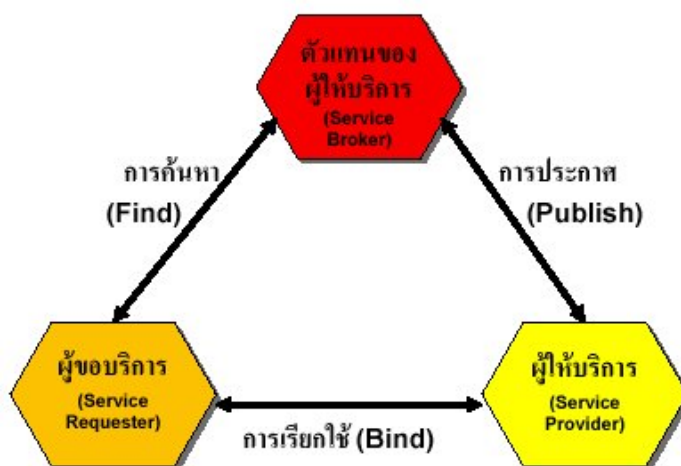
2.3 Service Oriented Architecture (SOA)

เว็บเซอร์วิสมีสถาปัตยกรรมแบบ Service Oriented Architecture (SOA) ซึ่งเป็นสถาปัตยกรรมที่เน้นการให้บริการ โดย SOA เป็นรูปแบบในการออกแบบระบบ (system) หรือ application แบบหนึ่ง อย่าง object-oriented เวลาออกแบบระบบเราจะมองเป็น object หรือ class แยกแยะระบบออกมาเป็น class และความสัมพันธ์ แต่ SOA จะมองระบบประกอบด้วยการทำงานหรือบริการ (service) ต่างๆ ซึ่งเซอร์วิสก็นับคล้ายกับฟังก์ชันหรือเมธอด กล่าวคือ มีหน้าที่ทำอะไรสักอย่างเมื่อมีการเรียกใช้ แต่เซอร์วิสมีลักษณะเป็น high-level และเป็นมุมมองจาก business จริงๆ มากกว่า เช่น มองระบบธนาคารประกอบด้วย withdrawal service, Deposit service แทนที่จะมองเป็น class Customer หรือ class Teller ที่มี operation withdrawal หรือ deposit นอกเหนือจากนั้นสิ่งที่เซอร์วิสทำได้ แต่เมธอดทั่วไปทำไม่ได้ คือ distributed system นั่นคือแต่ละ service สามารถถูกเรียกจากต่างระบบ, host หรือ JVM กันได้ และแต่ละ service มีอิสระต่อกัน

2.3.1 ส่วนประกอบเบื้องต้นของ SOA

SOA มีส่วนประกอบหลักด้วยกันสามส่วน คือ

- **Service Provider** (ผู้ให้บริการ) มีหน้าที่ในการสร้างบริการที่เป็นประโยชน์ และสร้างรายละเอียดของบริการนั้นๆ แล้วกระจายรายละเอียดนั้นออกไปยังระบบลงทะเบียนต่างๆ และรับการร้องขอบริการจากผู้ร้องขอบริการต่างๆ
- **Service Request** (ผู้ร้องขอบริการ) มีหน้าที่ในการค้นหาบริการที่อยู่ภายในระบบลงทะเบียนต่างๆ เพื่อนำรายละเอียดบริการมาร้องขอบริการที่กำหนดโดยผู้ให้บริการ
- **Service Broker** (ตัวแทนของผู้ให้บริการ) มีหน้าที่ในการกระจายข่าวสารรายละเอียดของบริการที่เกิดขึ้นจากผู้ให้บริการและอนุญาตให้ผู้ร้องขอบริการใช้การค้นหารายละเอียดของบริการที่อยู่ภายใน



รูปที่ 2.1 สถาปัตยกรรมแบบ SOA

องค์ประกอบทั้งสามจะทำงานร่วมกันโดยใช้ฟังก์ชันพื้นฐาน ดังนี้

- Publish (การประกาศ) เป็นการติดต่อระหว่างผู้ให้บริการกับตัวแทนของผู้ให้บริการ โดยผู้ให้บริการจะประกาศบริการของตนไว้กับตัวแทนของผู้ให้บริการ เพื่อให้ผู้ขอบริการมาทำการค้นหาบริการที่ต้องการได้
- Find (การค้นห) เป็นการติดต่อระหว่างผู้ขอบริการกับตัวแทนของผู้ให้บริการ โดยผู้ขอบริการจะใช้ขั้นตอนในการค้นหาเพื่อค้นหาบริการที่ตรงกับความต้องการ
- Bind (การเรียกใช้) เป็นการติดต่อระหว่างผู้ขอบริการกับผู้ให้บริการ โดยผู้ขอบริการจะทำการเรียกใช้บริการจากผู้ให้บริการ

2.4 เทคโนโลยีพื้นฐานของเว็บเซอร์วิส

Service Directory	: UDDI
Service Description	: WSDL
Service Interactive	: SOAP
Data Format	: XML
Communication Protocol	: HTTP
Communication Network	: Internet

2.4.1 XML (Extensible Markup Language)

XML เป็นภาษา Markup ที่เป็น text-based ซึ่งเป็นมาตรฐานที่ทุกระบบสนับสนุน ทำให้ข้อมูลที่มีโครงสร้างของภาษา XML จะถูกนำไปประมวลผลต่ออย่างอัตโนมัติได้อย่างง่ายดาย ภาษา XML จึงถูกนำมาใช้เป็นภาษามาตรฐานในการแลกเปลี่ยนข้อมูลของ Web Services

ลักษณะที่สำคัญของภาษา XML

- สิ่งที่แตกต่างกันระหว่าง XML และ HTML คือ XML ถูกออกแบบมาเพื่อใช้แลกเปลี่ยนข้อมูลไม่ใช่มาแทนที่ HTML ซึ่ง XML และ HTML ถูกออกแบบมาเพื่อจุดประสงค์ที่ต่างกัน คือ HTML จะเกี่ยวข้องกับการแสดงข้อมูล ส่วน XML จะเกี่ยวข้องกับการอธิบายข้อมูล
- XML ไม่มีแท็ก (tag) ที่สร้างขึ้น เราจะเป็นผู้กำหนดนิยาม และความหมายของแท็กตามข้อตกลงของ W3C ซึ่งทำให้การอ่านเอกสารเป็นไปได้ง่ายและเป็นมาตรฐาน
- XML สามารถรองรับการแลกเปลี่ยนข้อมูลระหว่างแอปพลิเคชัน โดยไม่ขึ้นอยู่กับแพลตฟอร์ม
- XML เป็นภาษาที่ได้รับการออกแบบมาเพื่อให้สามารถนิยามความหมายของข้อมูลได้ จึงมีการจัดโครงสร้างข้อมูล แบ่งข้อมูลออกเป็นหมวดหมู่ และส่วนประกอบย่อย
- ส่วนของข้อมูล และส่วนการแสดงผลของเอกสาร XML ถูกแยกออกจากกันอย่างชัดเจน โดยที่ในเอกสาร XML นั้นจะมีแต่ตัวเนื้อข้อมูล จึงทำให้การเปลี่ยนแปลงข้อมูลไม่ส่งผลกระทบต่อการแสดงผล และการแก้ไขส่วนแสดงผลก็จะไม่ส่งผลกระทบต่อตัวเนื้อข้อมูล

2.4.1.1 XML และ HTML

HTML เป็นภาษาที่ใช้ในการเขียน Web มากที่สุดนั่นเป็นเพราะมีรูปแบบที่ง่ายต่อการแสดงผลของ Browser เนื่องจาก มี tag ดาวยตัวที่สามารถบอกได้ว่าเมื่อเจอ tag นี้จะแสดงผลอย่างไร เช่น เมื่อเจอ tag ... ในเอกสารก็ให้แสดงข้อความที่อยู่ระหว่าง tag เป็นตัวหนา แต่จะสังเกตได้ว่าคอมพิวเตอร์จะไม่เข้าใจว่าข้อความนั้นคืออะไร เพียงแต่รู้ว่าจะแสดงผลอย่างไร นั่นแสดงว่าไม่สามารถนำข้อมูลภายใน tag เหล่านี้ไปทำการประมวลใดๆ ได้เลย

XML เป็นภาษาที่มีลักษณะเป็น tag คล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อความหมาย โดยอนุญาตให้ผู้ใช้สามารถกำหนด tag ขึ้นได้เอง เพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกัน ทำให้ข้อมูลระหว่าง tag สามารถนำไปประมวลผลต่อได้ เช่น

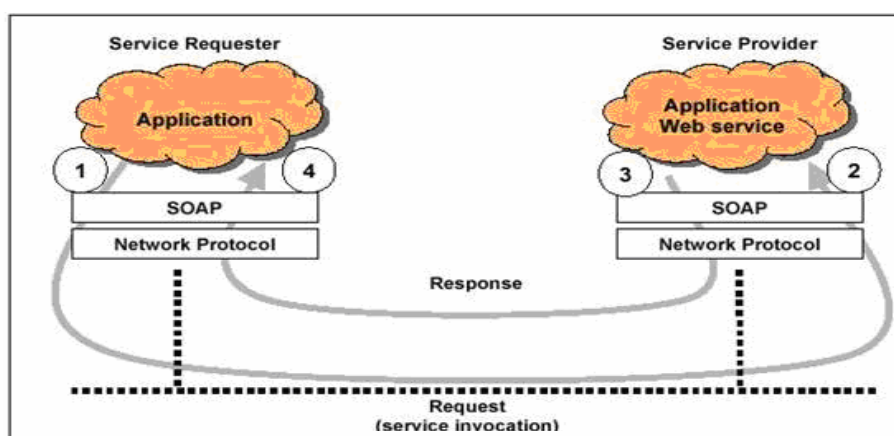
```
<?xml version="1.0" encoding="ISO-8859-1"?>
<note date = "12/11/2005">
  <to>Tove</to>
  <address>
    <street>123 Main St.</street>
    <city>San Francisco</city>
    <state>CA</state>
    <zip>94117</zip>
  </address>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

รูปที่ 2.2 ตัวอย่างเอกสาร XML

2.4.2 SOAP (Simple Object Access Control)

SOAP เป็นมาตรฐานที่ใช้เพื่อกำหนดการส่งข้อมูลของผู้ให้บริการและรับบริการ เป็นโปรโตคอลที่ผู้จัดทำ Web Services เลือกใช้ในการส่ง message ระหว่าง Web Services SOAP เป็น Transport Protocol ที่มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีในการเข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ไว้อย่างชัดเจน ในส่วนของการติดต่อไปยังคอมพิวเตอร์เครื่องหนึ่งและส่งผ่านข้อมูลไปให้ รวมถึงระบุวิธีที่โปรแกรมซึ่งถูกเรียกนั้นจะส่งค่าคืนกลับมาด้วย SOAP เป็น XML-based โปรโตคอล และใช้ HTTP เป็นโปรโตคอลร่วม สำหรับการแลกเปลี่ยนข้อมูล

SOAP ได้กำหนดเมสเสจจิงโปรโตคอล (Messaging Protocol) ระหว่างผู้ขอบริการกับผู้ให้บริการในการติดต่อสื่อสารกัน เช่น กำหนดให้ผู้ขอบริการต้องส่งข้อมูลที่ระบุฟังก์ชันและค่าพารามิเตอร์ต่างๆ ที่จำเป็นต้องใช้ในแอปพลิเคชันที่ร้องขอ ส่งไปให้กับผู้ให้บริการ ซึ่งแอปพลิเคชันของผู้ให้บริการก็จะทำงานตามกระบวนการที่ถูกร้องขอมา โดยมีการทำงานดังรูป



รูปที่ 2.3 XML messaging using SOAP

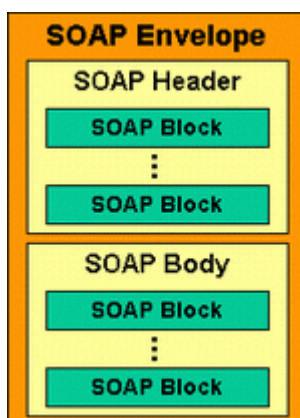
ขั้นตอนในการส่ง SOAP Message เพื่อทำการติดต่อ ระหว่างผู้เรียกใช้บริการกับผู้ให้บริการมีดังนี้

1. แอปพลิเคชันของผู้ขอบริการสร้าง SOAP Message เพื่อเรียกใช้บริการของเว็บเซอร์วิส
2. เว็บเซอร์วิสของผู้ให้บริการได้รับ SOAP Message จากผู้ร้องขอ ซึ่งอยู่ในรูปแบบ XML
3. เว็บเซอร์วิสประมวลผลตามคอมโพเนนต์ที่ให้บริการแล้วผู้ให้บริการก็จะสร้าง SOAP Message ที่มีผลลัพธ์นั้นกลับไปยังผู้ร้องขอบริการ
4. แอปพลิเคชันของผู้ขอบริการได้รับผลลัพธ์ที่เป็น SOAP Message แล้วทำการแปลงให้อยู่ในรูปแบบที่ต้องการ เพื่อนำไปประมวลผลต่อ

2.4.2.1 โครงสร้างของ SOAP

เอกสาร SOAP นั้นมีโครงสร้างในรูปแบบ XML ซึ่งเราสามารถแบ่งส่วนของเอกสารได้เป็น 3 ส่วนหลัก ดังนี้

1. SOAP envelop เนื้อหาสาระ (Content) ของเอกสารทั้งหมด
2. SOAP header ส่วนเพิ่มเติมของเอกสาร SOAP ซึ่งจะมีก็ได้ ไม่มีก็ได้
3. SOAP body ส่วนที่ใช้ในการเรียกใช้เซอร์วิส และผลลัพธ์ที่ได้จากเซอร์วิส



รูปที่ 2.4 โครงสร้างของเอกสาร SOAP

1. Envelope Element

SOAP เมสเสจทั้งหมดจะถูกห่อหุ้มในเอ็นวีโลบ (Envelope) ซึ่งมันจะเป็นอีลีเมนต์ที่อยู่บนสุดของ SOAP เมสเสจ นอกจากนี้ในส่วนของเอ็นวีโลบนั้นจะมีการใช้แอตทริบิวต์ encodingStyle ซึ่งจากตัวอย่างด้านล่างมีการอ้างการ encoding จาก

<http://www.w3.org/2001/12/soap-encoding>

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://schemas.xmlsoap.org/soap/envelope/">
  ...
  Message information goes here
  ...
</soap:Envelope>
```

รูปที่ 2.5 SOAP Envelope Element

2. Header Element

เฮดเดอร์อีลีเมนต์จะถูกใช้ในการส่งเมตาอินฟอร์เมชัน (meta information) ที่เกี่ยวกับ SOAP เมสเสจ แต่เมตาอินฟอร์เมชันนี้จะไม่มีความจำเป็นสำหรับส่วนของการเรียกใช้งานฟังก์ชัน ใน SOAP เมสเสจจะมีการกำหนดค่า mustUnderstand attribute ให้อยู่ในส่วนของ เฮดเดอร์อีลีเมนต์ ซึ่งจะใช้กรณีที่เซิร์ฟเวอร์เกิดไม่เข้าใจอีลีเมนต์นั้นก็จะมีการส่งเมสเสจแสดงความผิดพลาด (fault message) กลับมาบอก ซึ่งกระบวนการนี้จะทำให้เครื่องไคลเอนต์มั่นใจได้ว่า เครื่องเซิร์ฟเวอร์สามารถที่จะประมวลผลได้ โดยค่าแอททริบิวต์ตัวนี้มีค่าได้ 2 ค่า คือ 0 แสดงว่าไม่ต้องประเมินผล และ 1 แสดงว่าจำเป็นต้องมีการประเมินผล ดังนั้นตัวอย่าง ข้างล่างนี้จะใช้เฮดเดอร์สำหรับส่งค่าไอดี(ID) จากไคลเอนต์ไปยังเซิร์ฟเวอร์ ซึ่งเกี่ยวกับเรื่องระบบรักษาความปลอดภัย

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="
http://schemas.xmlsoap.org/soap/envelope/http://www.w3.org/2001/12/soap-
encoding">
<soap:Header>
<m:Trans
xmlns:m="http://www.w3schools.com/transaction/"
soap:mustUnderstand="1">234</m:Trans>
</soap:Header>
...
```

รูปที่ 2.6 SOAP Header Element

3. Body Element

เป็นส่วนที่ต้องมีอยู่ใน SOAP เมสเสจเสมอ และจะบรรจุใจความสำคัญของ SOAP เมสเสจ ซึ่งชนิดของเมสเสจนั้นจะบรรจุส่วนของการร้องขอใช้งานเมธอดโดยใช้ชื่อของเมธอดและพารามิเตอร์ของเมธอดนั้น ซึ่งจะอยู่ในรูปแบบของ XML โดยเฉพาะรูปแบบของพารามิเตอร์ที่ส่งจะเป็นส่วนสำคัญมากสำหรับการส่งข้อมูลที่ประมวลผลที่ได้จากผู้ให้บริการกลับมาในรูปแบบ XML และถ้าหากมีข้อผิดพลาดก็จะบอกใน ส่วนนี้

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body>
  <m:GetPrice
xmlns:m="http://www.w3schools.com/prices">
    <m:Item>Apples</m:Item>
  </m:GetPrice>
</soap:Body>
</soap:Envelope>
```

รูปที่ 2.7 ตัวอย่าง SOAP รีเควสต์เมสเสจ

```
<?xml version="1.0"?>
<soap:Envelope
xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body>
  <m:GetPriceResponse
xmlns:m="http://www.w3schools.com/prices">
    <m:Price>1.90</m:Price>
  </m:GetPriceResponse>
</soap:Body>
</soap:Envelope>
```

รูปที่ 2.8 ตัวอย่าง SOAP เรสพอนส์เมสเสจ

3.1 SOAP Fault Element

เป็นส่วนของที่จะถูกใช้เมื่อมีข้อผิดพลาดในการประมวลผลของเอกสาร SOAP ซึ่งโดยปกติจะเห็นเฉพาะที่เป็นเอกสารตอบกลับเท่านั้น (Reply Message หรือ Answer Message) โดยถ้ามี Fault Element ส่วนนี้จะอยู่ในตำแหน่งของ child element ของส่วน Body

SOAP Fault Element มี Sub Element ดังนี้

Sub Element	คำอธิบาย
<faultcode>	ชื่อของการเกิดข้อผิดพลาด (ดูตารางด้านล่าง)
<faultstring>	คำอธิบายของข้อผิดพลาด
<faultactor>	ชื่อของแหล่งที่เกิดข้อผิดพลาด
<detail>	ข้อมูลรายละเอียดเพิ่มเติมของข้อผิดพลาดนั้น

ตารางที่ 2.2 Sub Element ของ Fault Element

2.4.3 WSDL (Web Services Description Language)

WSDL นั้นเป็นภาษาที่ใช้ XML เป็นพื้นฐาน ซึ่งใช้ในการบรรยายเว็บเซอร์วิสหรือเน็ตเวิร์คเอ็นพอยต์ (network endpoint) เพื่อเป็นการเผยแพร่ข้อมูลการให้บริการแก่ระบบภายนอก โดยที่ทำงานผ่านระบบเครือข่าย โดยปราศจากการแทรกแซงของคน นอกจากนี้ WSDL นั้นยังสามารถที่จะบรรยายการส่งเมสเสจระหว่างเว็บเซอร์วิสระบุตำแหน่งที่อยู่ของเว็บเซอร์วิสและรวมไปถึงโปรโตคอลที่ใช้ในการติดต่อสื่อสารกันของเว็บเซอร์วิส WSDL นั้นจะทำงานร่วมกับ SOAP และ UDDI เพื่อที่จะทำให้เว็บเซอร์วิสติดต่อกับเว็บเซอร์วิสอื่นๆได้บนระบบอินเทอร์เน็ต ถ้าไม่มี WSDL แล้วการกระทำเมสเสจจึงอินเตอร์เฟส (messaging interface) นั้นจะต้องทำเอง WSDL คือ มาตรฐานสำหรับการประกาศโปรเซสที่จำเป็นใน การเรียกใช้เซอร์วิส

2.4.3.1 โครงสร้างเอกสาร WSDL

```

<definitions>
<types>
    definition of types.....
</types>
<message>
    definition of a message....
</message>
<portType>
    <operation>
        definition of a port.....
    </operation>
</portType>
<binding>
    <operation>...</operation>
    definition of a binding....
</binding>
<service>
    definition of a service....
</service>
</definitions>

```

รูปที่ 2.9 โครงสร้างเอกสาร WSDL

Element	Definition
Definitions	กำหนดรายละเอียดของเซอร์วิส โดยอีลีเมนต์นี้จะสนับสนุนแอทริบิวต์ (attributes) ต่างๆดังนี้ : • name (optional) • targetNamespace : เป็น logical namespace สำหรับข้อมูลเกี่ยวกับเซอร์วิส เอกสาร WSDL สามารถ import เอกสาร WSDL อื่น และกำหนด targetNamespace ให้เป็นค่าที่ unique เพื่อไม่ให้เกิดปัญหาNamespace ซ้ำกัน • xmlns : เป็น default namespace ของเอกสาร WSDL และจะถูกกำหนดค่าให้เป็น <code>http://schemas.xmlsoap.org/wsdl/</code> เพื่อใช้ WSDL elements มาตรฐาน เช่น <code><definitions></code>, <code><types></code> และ <code><message></code> ที่อยู่ใน namespace นี้ • xmlns:xsd และ xmlns:soap : เป็น standard namespace definitions ที่ถูกใช้สำหรับการระบุข้อมูล SOAP-specific เช่นเดียวกับ data types. • xmlns:tns หมายถึง <i>this namespace</i>.
Types	ให้ข้อมูลเกี่ยวกับ data types ใดๆที่ซับซ้อนซึ่งถูกใช้อยู่ในเอกสาร WSDL แต่ถ้าใช้ data types ที่ไม่ซับซ้อน เอกสาร WSDL ก็ไม่มีความจำเป็นที่จะต้องใช้ในส่วนนี้
message	อธิบาย data elements ของ operation แต่ละ message อาจมีมากกว่าหนึ่งส่วนเทียบได้กับ parameter ของ function ในการเขียนโปรแกรม
portType	เป็นส่วนที่สำคัญที่สุดใน WSDL element อธิบาย operations ที่ web service มีให้บริการและ messages ที่เกี่ยวข้อง เทียบได้กับ function library หรือ module หรือ class ในการเขียนโปรแกรม
operation	อธิบาย method ที่ให้บริการ Web Services หนึ่งจะมี method จำนวนกี่ method ก็ได้

binding	อธิบาย format ของ operation และ protocol details ในแต่ละ port
service	สำหรับ web server จะมี Web Services จำนวนกี่บริการก็ได้ และ ชื่อ Web Services ก็เป็นตัวจำแนกและบ่งบอกแต่ละบริการซึ่งห้ามมีชื่อซ้ำกัน ซึ่งภายใน service จะระบุ port address(es) ของการ binding โดย service จะเก็บรายละเอียดของ network endpoints หรือ ports.
port	กำหนด single endpoint ให้เป็นเหมือน address สำหรับ binding ดังนั้นมันจึงเป็นเหมือนตัวที่ใช้ในการกำหนด single communication endpoint.

ตารางที่ 2.3 คำอธิบายของแต่ละอีลีเมนต์ในเอกสาร WSDL

```

<?xml version = '1.0' encoding = 'windows-1252'?>
<!--Generated by the Oracle9i JDeveloper Web Services WSDL Generator-->
<!--Date Created: Wed Mar 06 12:59:39 IST 2002-->
<definitions
  name="CCServicesImpl"
  targetNamespace=
    "http://tempuri.org/jdeveloper/generated/oracle/otnsamples/ws/CCServicesImpl"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns=
    "http://tempuri.org/jdeveloper/generated/oracle/otnsamples/ws/CCServicesImpl"
  xmlns:ns1=
    "http://tempuri.org/jdeveloper/generated/oracle/otnsamples/ws/CCServicesImpl/schema">
  <types>
    <schema
      targetNamespace=
        "http://tempuri.org/jdeveloper/generated/oracle/otnsamples/ws/CCServicesImpl/schema"
      xmlns="http://www.w3.org/2001/XMLSchema"
      xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/" />
    </types>
    <message name="validateCard0Request">
      <part name="cardNumber"
        type="xsd:string"/>
    </message>
    <message name="validateCard0Response"/>
    <portType name="CCServicesImplPortType">
      <operation name="validateCard">
        <input name="validateCard0Request" message="tns:validateCard0Request"/>
        <output name="validateCard0Response" message="tns:validateCard0Response"/>
      </operation>
    </portType>
  </definitions>

```

```

<binding name="CCServicesImplBinding" type="tns:CCServicesImplPortType">
  <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="validateCard">
    <soap:operation soapAction="" style="rpc"/>
    <input name="validateCard0Request">
      <soap:body use="encoded" namespace="CCServices"
        encodingStyle="http://schemas.xmlsoap.org/soap/encoding/" />
    </input>

    <output name="validateCard0Response">
      <soap:body use="encoded" namespace="CCServices"
        encodingStyle="http://schemas.xmlsoap.org/soap/encoding/" />
    </output>
  </operation>
</binding>
<service name="CCServicesImpl">
  <documentation>
    Class providing credit card validation methods. This implementation
    uses LUHN algorithm to validate 16 digit number cards.
  </documentation>
  <port name="CCServicesImplPort" binding="tns:CCServicesImplBinding">
    <soap:address location="http://127.0.0.1:8988/vsm/CCServices"/>
  </port>
</service>
</definitions>

```

รูปที่ 2.10 ตัวอย่างเอกสาร WSDL

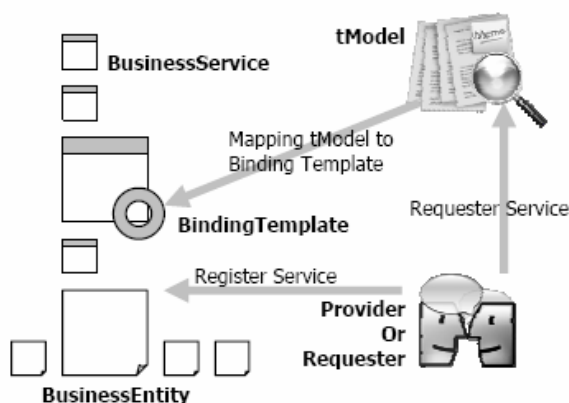
2.4.4 UDDI (Universal Description, Discovery, and Integration)

UDDI เป็นมาตรฐานที่ให้ชุดพื้นฐาน APIs (Application Programming Interface) ของ SOAP ที่สามารถนำมาใช้ในการพัฒนาเป็นตัวแทนของผู้ให้บริการ (Service broker) UDDI ใช้สำหรับค้นหาบริการ ที่ต้องการและเมื่อได้มาแล้ว UDDI ยังจัดหาข้อตกลงในวิธีการที่จะใช้งาน รูปแบบการทำงานของ UDDI จะเปลี่ยนแปลงวิธีการค้นหาข้อมูลบนอินเทอร์เน็ต ที่เราใช้อยู่ในปัจจุบันจากการพิมพ์ชื่อ โดเมนเนมในช่องแอดเดรสบาร์ ไปเป็นการใช้คำสามัญหรือชื่อเฉพาะในการค้นหา และเข้าสู่เว็บไซต์แทน UDDI จะทำหน้าที่เหมือนสมุดหน้าเหลืองที่จำแนกผู้จดทะเบียนตามรูปแบบของธุรกิจ แยกประเภทตามกิจการ

เมื่อมีผู้ที่ต้องการลงทะเบียนเว็บเซอร์วิส ในครั้งแรกจะต้องทำการลงทะเบียนเกี่ยวกับชื่อองค์กรผู้ให้บริการ, คำอธิบายรายละเอียดของเว็บเซอร์วิส, ข้อมูลการติดต่อ และหมายเลขอุตสาหกรรม จากนั้นจึงจะทำการลงทะเบียนเว็บเซอร์วิสได้ ซึ่งลงทะเบียนโดยระบุ URL ของ WSDL และคำอธิบายรายละเอียดเว็บเซอร์วิสนั้นๆ

2.4.4.1 โครงสร้างข้อมูลของ UDDI

UDDI ประกอบไปด้วย 2 ส่วน คือส่วนของรายละเอียดข้อมูลของผู้ให้บริการ หรือผู้ลงทะเบียนเว็บไซต์ ซึ่งจะอยู่ในรูปแบบของธุรกิจ (BusinessEntity) ที่อธิบายถึงบริการต่างๆ ที่มีอยู่ในธุรกิจนั้นๆ (BusinessService) โดยจะมีตัวแม่แบบ (BindingTemplate) ทำหน้าที่เชื่อมโยงไปยังส่วนที่สอง นั่นคือส่วนในการค้นหาที่เป็นรูปแบบของ tModel ซึ่งเป็นหัวใจหลักในการค้นหาข้อมูลที่ใช้มาตรฐาน UDDI



รูปที่ 2.11 ภาพรวมโครงสร้างข้อมูลของ UDDI

โครงสร้างต่างๆ จากรูปที่ 2.11 อธิบายได้ ดังนี้

BusinessEntity – เป็นโครงสร้างระดับบน เป็นตัวแทนของบริษัทที่ต้องการประกาศเผยแพร่เว็บไซต์ของเรา BusinessEntity จะมีข้อมูลเกี่ยวกับตัวบริษัท ตัวผู้ผลิต

(รายละเอียด, หมายเลข, และหมายเลขอุตสาหกรรม) และข้อมูลเกี่ยวกับ BusinessService ที่ตนให้บริการ

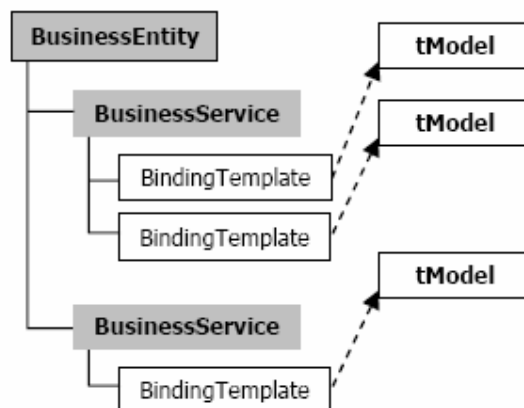
BusinessService – เป็นโครงสร้างซึ่งเป็นส่วนหนึ่งของ BusinessEntity ซึ่งหนึ่ง BusinessEntity อาจจะมีหลาย BusinessService โดย BusinessService จะเก็บรายละเอียดเกี่ยวกับตัวเว็บไซต์ของเราและหมายเลขของตัวเว็บไซต์ของเรารวมถึง BindingTemplate

BindingTemplate – เป็นข้อมูลที่อยู่ใน BusinessService มีหน้าที่เชื่อมโยงไปยัง tModel ซึ่งช่วยให้การค้นหาข้อมูลเร็วขึ้น

tModel – ถือเป็นหัวใจสำคัญของการค้นหาบน UDDI เพราะ tModel จะมีหน้าที่เป็น อินเด็กซ์ (index) ของ UDDI โดยจะมีนิยามประเภทของเว็บไซต์ของเราและอ้างอิงไปยังเอกสาร WSDL อีกด้วย

PublisherAssertion – เป็นคีย์อ้างอิงที่มีหน้าที่บอกความสัมพันธ์ระหว่างสอง Business Entity เพราะอาจจะเป็น บริษัทแม่-ลูกกัน (Parent-Child) หรือ พันธมิตรกัน (Peer2Peer)

หรืออื่นๆความสัมพันธ์ของโครงสร้าง UDDI ในการลงทะเบียนและการค้นหาเว็บเซอร์วิส ดังรูป



รูปที่ 2.12 ความสัมพันธ์ของโครงสร้างของ UDDI กับ tModel

2.5 ประเภทของเว็บเซอร์วิสและวิธีการเรียกใช้

เราสามารถแบ่งเว็บเซอร์วิสออกได้เป็น 2 ประเภท คือ

1. Atomic web service - เป็นเว็บเซอร์วิสทำงานด้วยตัวเองไม่จำเป็นต้องพึ่งพาเว็บเซอร์วิสอื่น ซึ่งจุดประสงค์โดยส่วนใหญ่ของการเรียกใช้เว็บเซอร์วิสประเภทนี้ คือ เรียกใช้เพราะเราต้องการใช้บริการจริงๆ ผู้เรียกมักเป็น end user client เช่น การที่ผู้ใช้ส่งข้อมูลไปยังเว็บเซอร์วิสของบริษัททัวร์ เพื่อซื้อ package ทัวร์
2. Composite web service – เป็นเว็บเซอร์วิสที่ต้องเรียกใช้เว็บเซอร์วิสอื่น เพื่อสร้างเซอร์วิสของตนเอง จุดประสงค์โดยส่วนใหญ่ของการเรียกใช้เว็บเซอร์วิสประเภทนี้ คือ เรียกใช้เพราะต้องการไปใช้เว็บเซอร์วิสอื่นต่อ ซึ่งก็คือ Composite Web Service นั่นเอง เช่น เว็บเซอร์วิสของบริษัททัวร์ เรียกใช้เว็บเซอร์วิสของโรงแรมเพื่อจองห้อง, สายการบินเพื่อจองตั๋ว และธนาคารเพื่อตัดยอดเงิน

การเรียกใช้เว็บเซอร์วิสมี 2 วิธีหลักๆ ได้แก่

1. ใช้ Web service client API เช่น JAVA proxy, WSIF, WSE เป็นต้น ก็คือเราเขียน application (ด้วย Java หรือ .NET) ของเราไปตามปกติ เมื่อถึงเวลาต้องเรียกใช้เว็บเซอร์วิสก็เรียกใช้ API แทน เหมือนกับการเขียนโปรแกรมปกติทั่วไป วิธีนี้เหมาะกับ end user client ที่เป็นผู้ใช้ปลายทางจริงๆ เพราะไม่ค่อยมีการเปลี่ยนแปลงบ่อยและมักเรียกเพียงไม่กี่เซอร์วิส อย่างไรก็ตามวิธีนี้ไม่เหมาะกับการสร้าง Composite Web Service เพราะเรามักใช้ Composite Web Service เพื่อสร้าง business process (business process คือ บริการที่เห็นหรือสัมผัสได้โดยตรงจากผู้ใช้หรือลูกค้า และให้ผลตอบแทนกับองค์กร นั่นก็คือ service นอกสุดที่ให้บริการลูกค้า

โดยตรงนั่นเอง) ซึ่งมี business logic จึงอาจจะเกิดการเปลี่ยนแปลงได้บ่อยๆ นอกจากนี้ business goal ของการสร้าง application ด้วยเว็บเซอร์วิส ก็เพื่อความคล่องตัว (agility) ปรับเปลี่ยนง่าย จะได้สอดคล้องกับสภาพธุรกิจปัจจุบันที่มีการแข่งขันสูง ดังนั้นการผูกแต่ละเว็บเซอร์วิสไว้ด้วยการcoding แบบเก่านั้น จึงไม่เหมาะกับการสร้าง business process ด้วยเว็บเซอร์วิสตามหลักการของ SOA

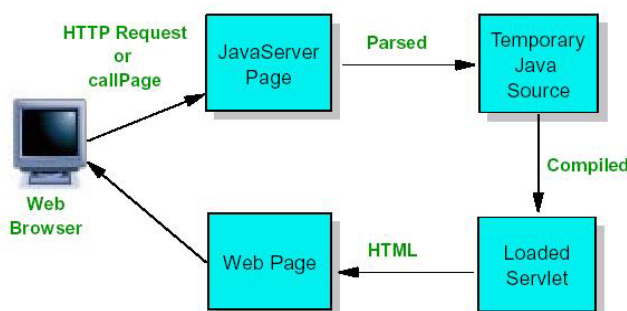
2. ใช้เว็บเซอร์วิสตัวกลาง (mediator) มาเรียกใช้ Sub-Web Service แต่วิธีการนี้ไม่เหมาะสมตามที่ได้อธิบายแล้วข้างต้น จึงมีการนำ BPEL (Business Process Execution Language) มาใช้เพื่อสร้าง business process หรือ composite Web Service นั่นเอง

2.6 JavaServer Pages (JSP)

JavaServer Pages หรือ JSP นั้นมีการทำงานที่อิงบนพื้นฐานของเทคโนโลยีจาวาที่สร้างโค้ดออกมาในรูปแบบของไดนามิกเว็บเพจ โดยสามารถทำงานได้บนทุกแพลตฟอร์มและบนเว็บเซิร์ฟเวอร์

JSP นั้นมีการเขียนคล้ายกับภาษา Active Server Pages (ASP) ของทางไมโครซอฟท์ แต่ใช้งานร่วมกับคำสั่ง HTML และจาวาโค้ดที่คล้ายกับ VBScript แต่ภาษา JSP นั้นจะทำงานบนเว็บเซิร์ฟเวอร์ที่ ASP ไม่สามารถทำงานได้ เช่นเซิร์ฟเวอร์อย่าง Apache หรือ Netscape ซึ่งไลเซนส์ของ JSP ถูกพัฒนาโดยทาง Sun ซึ่งสามารถใช้งานได้ฟรี ซึ่งสามารถทำงานได้บนระบบปฏิบัติการ Solaris, Linux และบนวินโดวส์

JSP นั้นจะทำงานทางฝั่งของเซิร์ฟเวอร์คล้ายกับ จาวาสคริปต์ อาจเรียกได้ว่าเป็นการทำงานแบบ Server side ขั้นตอนการทำงานจะเริ่มตั้งแต่การร้องขอ หรือ เกิด Request จาก Browser หรือ Client มาที่ JSP บนฝั่ง Server จากนั้น Server ก็ทำการประมวลผล JSP เป็น servlet ก่อน แล้วส่ง Response กลับไปให้ Client ในรูปของ HTML ดังนั้นภาษา JSP จึงสามารถควบคุมความสามารถได้มากกว่าภาษาอื่นๆที่ออกมาก่อนหน้านี้เช่น Perl scripts หรือ API ทางฝั่งเซิร์ฟเวอร์อย่าง ASP เป็นต้น



รูปที่ 2.13 การทำงานของ JavaServer Pages (JSP)

2.6.1 ตัวอย่างการเขียน JSP

```
File test.jsp
<html><body>
<%
JSP Command1;
JSP Command2;
JSP Command3;
%>
</body>
</html>
```

JSP มีลักษณะรูปแบบการเขียนเป็น HTML ผสมกับ tag ของจาวาแทรกกัน และไฟล์ที่เขียนด้วยเทคโนโลยีนี้ จะต้องกำหนดให้ไฟล์มีนามสกุลเป็น *.jsp เสมอ เพื่อบอกให้ server ทราบว่าจะต้องทำการประมวลผลกับไฟล์ที่ถูกร้องขออย่างไร การทำงานเมื่อทำการประมวลผล jsp ด้วย browser ใดๆ จะยังไม่สามารถทำการแสดงผลได้ในทันที แต่ jsp จะถูกทำการประมวลผลเป็น servlet ก่อน จากนั้น server จะทำการส่งผลลัพธ์ออกมาให้ browser เพื่อทำการแสดงผลอีกทีหนึ่ง (จะเห็นว่าการทำงานจะแตกต่างกับ HTML คือ JSP จะไม่สามารถทำการแสดงผลได้ในทันที แต่จะต้องมีการประมวลผลบน server ก่อน)

2.6.2 เปรียบเทียบ JSP กับเทคโนโลยีอื่นๆ

2.6.2.1 แอคทีฟเซิร์ฟเวอร์เพจ

แอคทีฟเซิร์ฟเวอร์เพจเป็นเทคโนโลยีที่ใช้หลักการคล้ายกับ JSP แต่เป็นเทคโนโลยีที่พัฒนาโดยบริษัทไมโครซอฟต์ โดยจะมีรูปแบบที่แตกต่างกันอย่างเด่นชัด 2 ประการคือ

ประการแรกภาษาที่ใช้ใน JSP คือภาษาจาวา ส่วนภาษาที่ใช้ใน ASP คือภาษา VisualBasic หรือภาษาอื่นๆ ที่กำหนดโดยไมโครซอฟต์

ประการที่สองเป็นผลสืบเนื่องมาจากประการแรก คือ เนื่องจากใช้ภาษาจาวาทำให้ JSP สามารถนำไปใช้งานบนระบบปฏิบัติการใดก็ได้ แต่ ASP ต้องใช้ในระบบปฏิบัติการที่ไมโครซอฟต์กำหนดเท่านั้น

2.6.2.2 จาวาเซิร์ฟเล็ต

ตัว JSP สามารถทำงานได้เช่นเดียวกับที่เซิร์ฟเล็ตทำได้ แต่จุดเด่นที่สำคัญของ JSP คือ สะดวกในการสร้างและเปลี่ยนแปลงมากกว่า เพราะสามารถแก้ไขที่ไฟล์ได้โดยตรง และไม่จำเป็นต้องคอมไพล์ใหม่ก่อนนำไปใช้งาน

2.6.2.3 จาวาสคริปต์

เป็นการทำเนื้อหาไดนามิกให้กับเว็บเหมือนกัน แต่จาวาสคริปต์จะเกิดการประมวลผลที่ไคลเอนต์ ถ้าเว็บเบราว์เซอร์ไม่สนับสนุนจาวาสคริปต์ก็จะทำให้ไม่สามารถทำงานได้ แต่ JSP จะเกิดการประมวลผลที่ตัวเซิร์ฟเวอร์โดยตรงทำให้ไม่มีปัญหากับตัวเบราว์เซอร์